

RTL8762E Keyboard Application Design Spec

V1.0

2021/8/24

Revision History

Date	Version	Comments	Author	Reviewer
2021/8/24	Draft			

Contents

Revision History	2
Table List	4
1 Overview	6
1.1 Device List	6
1.2 System Requirement	6
1.3 Terminology Definition.....	6
2 Software Structure	7
2.1 Key Module.....	8
2.2 Keyscan Module.....	9
2.3 APP Task Message Handler	12
2.4 Pair and Reconnect Process.....	12
3 Bluetooth Related Operations.....	14
3.1 Service and Characteristic	14
3.1.1 HID Service	14
3.1.2 Protocol Mode Characteristic	15
3.1.3 Report Characteristic	15
3.1.4 Report Map Characteristic.....	15
3.1.5 Boot Key Input Report Characteristic	16
3.1.6 Boot Key Output Report Characteristic.....	16
3.1.7 HID Information Characteristic.....	16
3.1.8 HID Control Point Characteristic	17
3.1.9 Battery Service	17
3.1.10 Battery Service Characteristic	17
3.1.11 Device Information Service	18
3.1.12 Device Information Service Characteristic.....	18
3.2 Report Map and Data Sending Format.....	19
3.3 Scan	22
4 Reference	23

Figure List

Figure 2.1	Software Structure.....	7
Figure 2.2	Key press handler process.....	9
Figure 2.3	APP Task Handler for IO Messages.....	12

Table List

Table 3.1 Service name and corresponding UUID	14
Table 3.2 HID Service Characteristic List	14
Table 3.3 Protocol Mode Characteristic Value Format	15
Table 3.4 Report Characteristic Value Format	15
Table 3.5 Report Map Characteristic Value Format	15
Table 3.6 Boot Mouse Input Report Characteristic Value Format	16
Table 3.7 HID Information Characteristic Value Format	16
Table 3.8 Bit Field	16
Table 3.9 HID Control Point Characteristic Value Format	17
Table 3.10 Battery Service Characteristic List	17
Table 3.11 Battery Level Characteristic Value Format	17
Table 3.12 Device Information Service Characteristic List	18
Table 3.13 Device Information Characteristic Value Format	18
Table 3.14 System ID Characteristic Value Format	19
Table 3.15 IEEE 11073-20601 Regulatory Certification Data List Characteristic Value Format	19
Table 3.16 PnP ID Characteristic Value Format	19
Table 3.17 Enumerations	19

1 Overview

1.1 Device List

1. Bee3 Evaluation Board
2. 4*4 Matrix Keyboard

1.2 System Requirement

Tools that need to be downloaded and installed on PC:

1. Keil MDK-ARM
2. SEGGER's J-Link tools
3. RTL8762E SDK
4. RTL8762E Flash programming algorithm

1.3 Terminology Definition

1. DLPS: Deep Low Power State
2. SUT: System Under Test, Android or IOS mobile devices with Keyboard application
3. HID: Human Interface Device protocol

2 Software Structure

Keyboard application mainly interacts with IO Driver and BT to accomplish specific function. IO module includes Keyscan module and GPIO key module. Single GPIO module is used to pair key. The software structure is shown below:

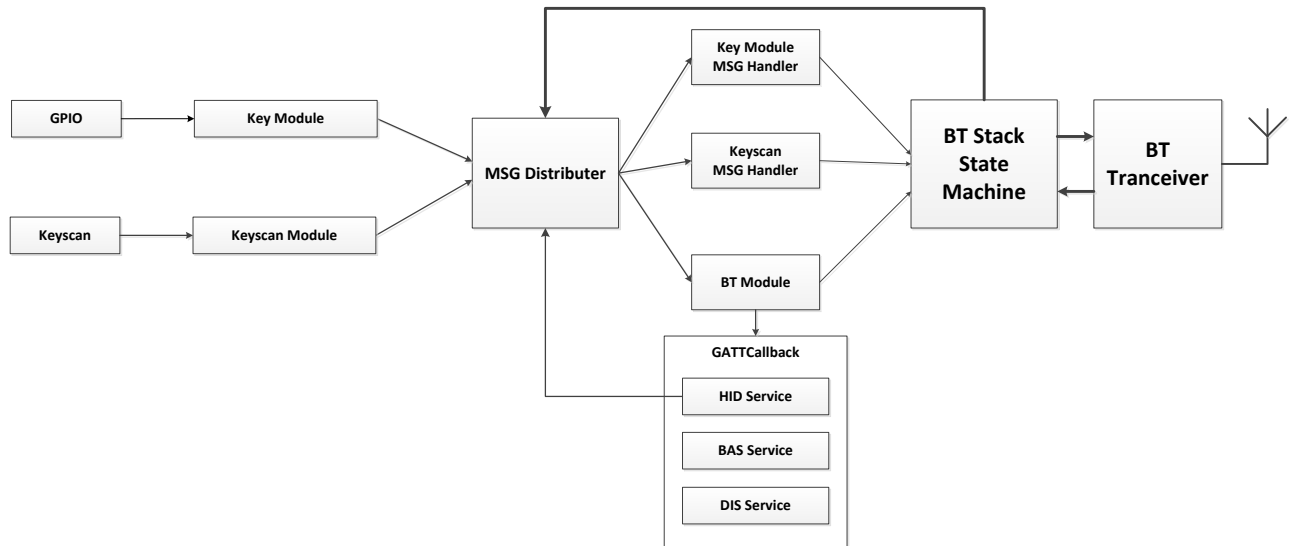


Figure 2.1 Software Structure

After booting and initializing system, APP task starts to run and wait for messages from GPIO key, keyscan and upper stack. To simplify handling of various types of message, App Task will create an Event Queue to store type of external message instead of receiving message directly. Then APP Task acquires information from corresponding queue based on message type stored in Event Queue and performs further operation.

```

1. while (true)
2. {
3.     if (os_msg_rcv(evt_queue_handle, &event, 0xFFFFFFFF) == true)
4.     {
5.         if (event == EVENT_IO_TO_APP)
6.         {
7.             T_IO_MSG io_msg;
8.             if (os_msg_rcv(io_queue_handle, &io_msg, 0) == true)
9.             {
10.                app_handle_io_msg(io_msg);
11.            }
12.        }
13.        else
14.        {
15.            gap_handle_msg(event);
16.        }

```

```
17.    }  
18. }
```

2.1 Key Module

Pairing key of Keyboard application is accomplished by applying single GPIO. If not paired, long press pair key to send pairing advertising. If paired, long press pair key to clear pairing information and resend pairing advertising.

GPIO connected to key needs to be set to edge trigger mode. When configuring GPIO interrupt, the trigger polarity should be set to the opposite of the current level. When key is pressed to trigger GPIO interrupt, system reads key status in interrupt handler function, flip the polarity that triggers the GPIO interrupt and send message to App Task.

Keyscan mode have two debounce mechanism to dealing with Active Mode and Sleep Mode respectively: Keyscan HW Debounce and PAD HW Debounce .

1. System in Active mode: Keyscan mode is in working mode, Keyscan HW Debounce is used .
2. System in Sleep mode: Keyscan mode is not in working mode, PAD HW Debounce is used. In the debounce stage, the system can also stay in low power consumption mode .

Long press can be implemented with timer. After key press event happens, APP Task receives key press message and start timer. If task does not receive key release event before timer is timeout, task will handle key long press event in timeout callback function. Otherwise, if key is released before timer is timeout, timer will be turned off.

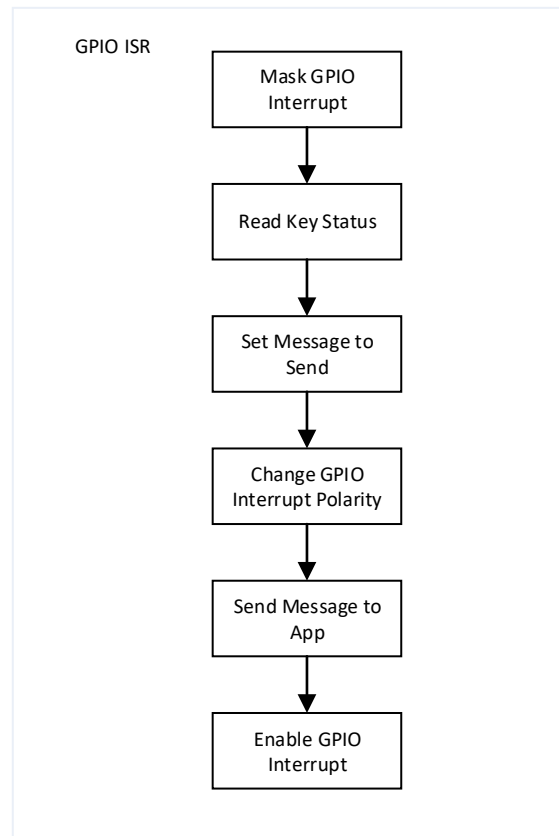


Figure 2.2 Key press handler process

2.2 Keyscan Module

Keyboard application uses Keyscan module to scan matrix keyboard. When key press event is scanned, message is sent to APP through keyscan interrupt.

```

1. void keyscan_interrupt_handler(void)
2. {
3.     APP_PRINT_INFO0("[keyscan_interrupt_handler] interrupt handler");
4.
5.     T_IO_MSG bee_io_msg;
6.     bee_io_msg.type = IO_MSG_TYPE_KEYSCAN;
7.
8.     if (KeyScan_GetFlagState(KEYSCAN, KEYSCAN_INT_FLAG_SCAN_END) == SET)
9.     {
10.        keyscan_global_data.is_allowed_to_enter_dlps = true;
11.
12.        memset(&keyscan_global_data.cur_fifo_data, 0, sizeof(T_KEYSCAN_FIFO_DATA));
13.
14.        /* read keyscan FIFO data length */
15.        keyscan_global_data.cur_fifo_data.len = KeyScan_GetFifoDataNum(KEYSCAN);
  
```

```

16.     if (keyscan_global_data.cur_fifo_data.len != 0)
17.     {
18.         keyscan_global_data.is_all_key_released = false;
19.
20.         /* read keyscan fifo data */
21.         KeyScan_Read(KEYSCAN, (uint16_t *) &(keyscan_global_data.cur_fifo_data.key[0]),
22.             keyscan_global_data.cur_fifo_data.len);
23.
24.         KeyScan_INTConfig(KEYSCAN, KEYSCAN_INT_SCAN_END, DISABLE);
25.         KeyScan_Cmd(KEYSCAN, DISABLE);
26.
27.         /* start sw timer to check press status */
28.         if (!os_timer_restart(&keyscan_timer, KEYSCAN_SW_INTERVAL))
29.         {
30.             APP_PRINT_ERROR0("[keyscan_interrupt_handler] restart keyscan_timer
failed!");
31.             /* set flag to default status and reinit keyscan module with debounce enabled
*/
32.             keyscan_init_data();
33.             keyscan_init_driver(KeyScan_Manual_Sel_Key, KeyScan_Debounce_Enable);
34.             return;
35.         }
36.
37.         if (false == keyscan_global_data.is_allowed_to_repeat_report)
38.         {
39.             if (!memcmp(&keyscan_global_data.cur_fifo_data,
&keyscan_global_data.pre_fifo_data,
40.                 sizeof(T_KEYSCAN_FIFO_DATA)))
41.             {
42.                 /* some keyscan FIFO data, just return */
43.                 return;
44.             }
45.             else
46.             {
47.                 /* update previous keyscan FIFO data */
48.                 memcpy(&keyscan_global_data.pre_fifo_data,
&keyscan_global_data.cur_fifo_data,
49.                     sizeof(T_KEYSCAN_FIFO_DATA));
50.             }
51.         }
52.
53.         bee_io_msg.subtype = IO_MSG_KEYSCAN_RX_PKT;
54.         bee_io_msg.u.buf = (void *)&keyscan_global_data.pre_fifo_data;
55.         if (false == app_send_msg_to_apptask(&bee_io_msg))

```

```

56.         {
57.             APP_PRINT_ERROR0("[keyscan_interrupt_handler] send IO_MSG_KEYSCAN_RX_PKT
message failed!");
58.             /* set flag to default status and reinit keyscan module with debounce enabled
*/
59.             keyscan_init_data();
60.             os_timer_stop(&keyscan_timer);
61.             keyscan_init_driver(KeyScan_Manual_Sel_Key, KeyScan_Debounce_Enable);
62.             return;
63.         }
64.     }
65.     else
66.     {
67.         if (false == keyscan_global_data.is_all_key_released)
68.         {
69.             /* keyscan release event detected */
70.             APP_PRINT_INFO0("[keyscan_interrupt_handler] keyscan release event
detected");
71.             keyscan_global_data.is_all_key_released = true;
72.
73.             bee_io_msg.subtype = IO_MSG_KEYSCAN_ALLKEYRELEASE;
74.             if (false == app_send_msg_to_apptask(&bee_io_msg))
75.             {
76.                 APP_PRINT_ERROR0("[keyscan_interrupt_handler] Send IO_MSG_TYPE_KEYSCAN
message failed!");
77.
78.                 keyscan_init_data();
79.                 keyscan_init_driver(KeyScan_Manual_Sel_Key, KeyScan_Debounce_Enable);
80.                 return;
81.             }
82.         }
83.
84.         os_timer_stop(&keyscan_timer);
85.         /* reinit keyscan module to key trigger manual mode with debounce enabled */
86.         memset(&keyscan_global_data.pre_fifo_data, 0, sizeof(T_KEYSCAN_FIFO_DATA));
87.         keyscan_init_driver(KeyScan_Manual_Sel_Key, KeyScan_Debounce_Enable);
88.         return;
89.     }
90. }
91. else
92. {
93.     /* if not KEYSCAN_INT_FLAG_SCAN_END interrupt */
94.     APP_PRINT_INFO0("[keyscan_interrupt_handler] not KEYSCAN_INT_FLAG_SCAN_END
interrupt");

```

```

95.     keyscan_init_data();
96.     /* start key trigger keyscan manual mode */
97.     keyscan_init_driver(KeyScan_Manual_Sel_Key, KeyScan_Debounce_Enable);
98.     return;
99. }
100. }

```

2.3 APP Task Message Handler

GPIO key module and Keyscan module send message to APP task, and APP task invokes different functions to handle message according to message type. At last, send data in the format defined in HID Report map (please refer to chapter 3.2) to SUT

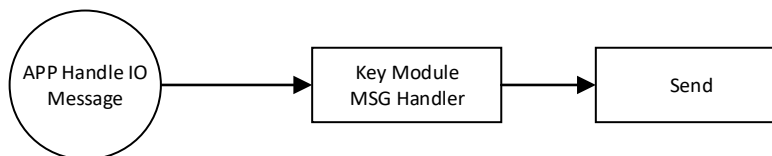


Figure 2.3 APP Task Handler for IO Messages

2.4 Pair and Reconnect Process

Device is not in pair mode when first used. User can long press pair key to trigger pairing mode, then system will send undirected advertising. After host device scan and find the Keyboard device, it can start pairing operation. Device will be added to resolving list and white list after paired. The resolving list is used to resolve Resolvable Private Address.

When reconnecting, APP will send different reconnection advertising type based on different device address type. If address type is public address or static random address, APP will send directed advertising. If address type is resolvable private address, APP will send undirected advertising and use white list to filter. After configuring resolving list and enable resolving, APP can filter device for resolvable private address, for RTL8762E supports LL Privacy.

Besides, when paired device resets, APP will check if pair information exists. If pair information exists, add corresponding device to resolving list and white list. Otherwise, device is not paired.

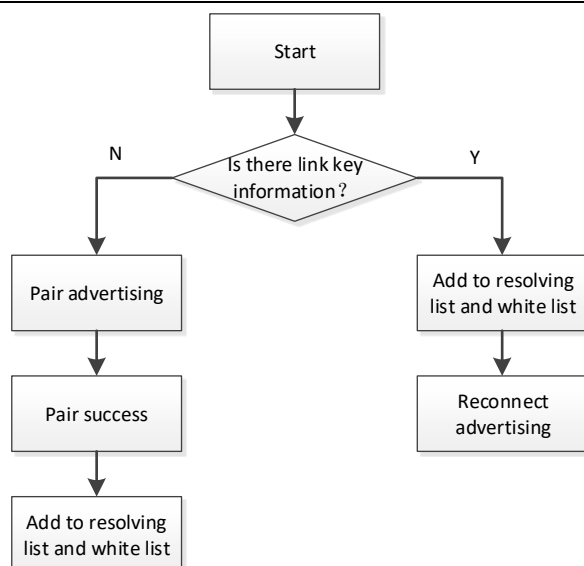


Figure 2.4 Pair and Reconnect Process

3 Bluetooth Related Operations

3.1 Service and Characteristic

Keyboard application contains the following services:

1. HID Service: human interface device protocol
2. Battery Service: report battery level of device, remind user to change battery, and prohibit OTA when battery level is low.
3. Device Information Service: display device information.

Service name and corresponding UUID are shown below

Table 3.1 Service name and corresponding UUID

Service Name	Service UUID
HID Service	0x1812
Battery Service	0x180F
Device Information Service	0x180A

3.1.1 HID Service

Table 3.2 HID Service Characteristic List

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Protocol Mode	M	0x2A4E	Read/WriteWithoutResponse	See Protocol Mode
Report	O			
Report:Input	M	0x2A4D	Read/Write/Notify	See Report Characteristic
Report:Output	M	0x2A4D	Read/Write/WriteWithoutResponse	See Report Characteristic
Report:Feature	M	0x2A4D	Read/Write	See Report Characteristic
Report Map	M	0x2A4B	Read	See Report Map
Boot Key Input Report	M	0x2A22	Read/Write/Notify	See Boot Key Input Report
Boot Key Output Report	M	0x2A32	Read/Write/WriteWithoutResponse	See Boot Key Output Report

HID Information	M	0x2A4A	Read	See HID Information
HID Control Point	M	0x2A4C	WriteWithoutResponse	See HID Control Point
Report: Input(For multimedia key)	M	0x2A4D	Read/Write/Notify	See Report Characteristic

3.1.2 Protocol Mode Characteristic

Protocol Mode Characteristic indicates protocol mode for current HID service, and set protocol mode for expected HID service

Table 3.3 Protocol Mode Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information	
Protocol Mode	Mandatory	uint8	N/A	N/A	Enumerations	
					Key	Value
					0	Boot Protocol Mode
					1	Report Protocol Mode
					2-255	Reserved for future use

3.1.3 Report Characteristic

Report Characteristic includes data in input report, output report and feature report transmitted between HID device and host device

Table 3.4 Report Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Report	Mandatory	uint8	N/A	N/A	This field may be repeated

3.1.4 Report Map Characteristic

Report Map Characteristic is used to define data format in input report, output report and feature report transmitted between HID device and host device.

Table 3.5 Report Map Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Report Map	Mandatory	uint8	N/A	N/A	This field may be repeated

3.1.5 Boot Key Input Report Characteristic

Boot Key Input Report Characteristic is used to boot the HID host device operated in protocol mode and input report with fixed format and length transmitted between HID host device and keyboard.

Table 3.6 Boot Mouse Input Report Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Boot Mouse Input Report	Mandatory	uint8	N/A	N/A	This field may be repeated

3.1.6 Boot Key Output Report Characteristic

Boot Key Input Report Characteristic is used to boot the HID host device operated in protocol mode and output report with fixed format and length transmitted between HID host device and keyboard.

Table 3.7 Boot Mouse Input Report Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Boot Key Output Report	Mandatory	uint8	N/A	N/A	This field may be repeated

3.1.7 HID Information Characteristic

HID Information Characteristic contains HID attributes. The characteristic value is static and can be permanently saved for bonding HID device and host device

Table 3.8 HID Information Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
bcdHID	Mandatory	uint16	N/A	N/A	None
bCountryCode	Mandatory	8bit	N/A	N/A	None
Flags	Mandatory	8bit	N/A	N/A	See Bit Field

Table 3.9 Bit Field

Bit	Size	Name	Key	Definition Value
0	1	Remote Wake	0	The device is not designed to be capable of

			providing wake-up signal to a HID host	
			1	The device is designed to be capable of providing wake-up signal to a HID host
1	1	Normally Connectable	0	The device is not normally connectable
			1	The device is normally connectable
2	6	Reserved for future use		

3.1.8 HID Control Point Characteristic

HID Control Point Characteristic is a control point attribute which defines the following HID commands:

1. Suspend
2. Exit Suspend

Table 3.10 HID Control Point Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information	
						Enumerations
HID Control Point Command	Mandatory	uint8	N/A	N/A	Key	Value
					0	Suspend
					1	Exit Suspend
					2-255	Reserved for future use

3.1.9 Battery Service

Battery Service includes a battery level characteristic, shown in Table 3.10

Table 3.11 Battery Service Characteristic List

Characteristic Name	Requirement	Characteristic UUID	Properties	Description
Battery Level	M	0x2A19	Read/Notify	See Battery Level

3.1.10 Battery Service Characteristic

Battery level represents current power level which ranges from 0 to 100 percent. Its data format is unsigned 8-bit integer, as shown in Table 3.11

Table 3.12 Battery Level Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
-------	-------------------	--------	---------------	---------------	------------------------

Battery Level	Mandatory	uint8	0	100	Enumerations	
					Key	Value
					101-255	Reserved

3.1.11 Device Information Service

Device Information Service includes 9 characteristics, shown in Table 3.12

Table 3.13 Device Information Service Characteristic List

Characteristic Name	Requirement	Characteristic UUID	Properties
Manufacturer Name String	O	0x2A29	Read
Model Number String	O	0x2A24	Read
Serial Number String	O	0x2A25	Read
Hardware Revision String	O	0x2A27	Read
Firmware Revision String	O	0x2A26	Read
Software Revision String	O	0x2A28	Read
System ID	O	0x2A23	Read
Regulatory Certification Data List	O	0x2A2A	Read
PnP ID	O	0x2A50	Read

3.1.12 Device Information Service Characteristic

Device Information Service contains characteristics of displaying basic information on device name and firmware version, shown in Table 3.13

Table 3.14 Device Information Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Manufacturer Name	Mandatory	utf8s	N/A	N/A	None
Model Number	Mandatory	utf8s	N/A	N/A	None
Serial Number	Mandatory	utf8s	N/A	N/A	None
Hardware Revision	Mandatory	utf8s	N/A	N/A	None
Firmware Revision	Mandatory	utf8s	N/A	N/A	None
Software Revision	Mandatory	utf8s	N/A	N/A	None

1) System ID Characteristic

System ID consists of two fields, 40-bit vendor-defined ID and 24-bit OUI (Organization Unique Identity), as

shown in Table 3.14

Table 3.15 System ID Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Manufacturer Identifier	Mandatory	uint40	0	1099511627775	None
Organization Unique Identifier	Mandatory	uint24	0	16777215	None

2) IEEE 11073-20601 Regulatory Certification Data List Characteristic

IEEE 11073-20601 Regulatory Certification Data List lists different certifications that device need to comply with.

Table 3.16 IEEE 11073-20601 Regulatory Certification Data List Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Data	Mandatory	reg-cert-data-list ^[1]	N/A	N/A	None

3) PnP ID Characteristic

PnP ID is a group of ID for identifying unique device, including Vendor ID source, Vendor ID and Product Version. These values are used to identify specific type/mode/version of device, as shown in Table 3.16.

Table 3.17 PnP ID Characteristic Value Format

Names	Field Requirement	Format	Minimum Value	Maximum Value	Additional Information
Vendor ID Source	Mandatory	uint8	1	2	See Enumerations
Vendor ID	Mandatory	uint16	N/A	N/A	None
Product ID	Mandatory	uint16	N/A	N/A	None
Product Version	Mandatory	uint16	N/A	N/A	None

Table 3.18 Enumerations

Key	1	2	3-255	0
Value	Bluetooth SIG assigned Company Identifier value from the Assigned Numbers document	USB Implementer's Forum assigned Vendor ID value	Reserved for future use	Reserved for future use

3.2 Report Map and Data Sending Format

Report Map is used to define data format for transporting input report, output report and feature report data

between HID device and host device.

Keyboard includes modifier keys and common keys. The press and release status for each modifier key is represented by 1 bit. Common keys are pressed to send its corresponding usage ID and at most 6 key values can be sent at the same time.

Keyboard data format:

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7
Modifier Keys	Reserved	Keycode1	Keycode2	Keycode3	Keycode4	Keycode5	Keycode6

Multimedia keys have different report ID, and each has its own data format. HID Keyboard profile in SDK lists 24 Multimedia keys in enum format. Sending data length is 3 bytes, and each bit corresponds to a multimedia key.

Multimedia key data format:

Byte	Bit							
	7	6	5	4	3	2	1	0
1	Scan Next Track	Scan Previous Track	Stop	Play/Pause	Mute	Bass Boost	Loudness	Volume Increment
2	Volume Decrement	Bass Increment	Bass Decrement	Treble Increment	Treble Decrement	AL Consumer Control Configuration	AL Email Reader	AL Calculator
3	AL Local Machine Browser	AC Search	AC Home	AC Back	AC Forward	AC Stop	AC Refresh	AC Bookmarks

Take data 00000001 00000000 00100000 as example, it represents Volume increment and Home keys pressed (refer to bit sequence of report map)

```

1. const uint8_t hids_report_descriptor[] =
2. {
3.     0x05, 0x01, /* USAGE_PAGE (Generic Desktop) */
4.     0x09, 0x06, /* USAGE (Keyboard) */
5.     0xa1, 0x01, /* COLLECTION (Application) */
6.     0x85, HOGP_KB_REPORT_ID, /* REPORT_ID (3) */
7.     0x05, 0x07, /* USAGE_PAGE (Keyboard) */
8.     0x19, 0xe0, /* USAGE_MINIMUM (Keyboard Left Control) */
9.     0x29, 0xe7, /* USAGE_MAXIMUM (Keyboard Right GUI) */
10.    0x15, 0x00, /* LOGICAL_MINIMUM (0) */

```

```

11. 0x25, 0x01, /* LOGICAL_MAXIMUM (1) */
12. 0x75, 0x01, /* REPORT_SIZE (1) */
13. 0x95, 0x08, /* REPORT_COUNT (8) */
14. 0x81, 0x02, /* INPUT (Data,Var,Abs) */
15. 0x95, 0x01, /* REPORT_COUNT (1) */
16. 0x75, 0x08, /* REPORT_SIZE (8) */
17. 0x81, 0x01, /* INPUT (Cnst,Var,Abs) */
18. 0x95, 0x05, /* REPORT_COUNT (5) */
19. 0x75, 0x01, /* REPORT_SIZE (1) */
20. 0x05, 0x08, /* USAGE_PAGE (LEDs) */
21. 0x19, 0x01, /* USAGE_MINIMUM (Num Lock) */
22. 0x29, 0x05, /* USAGE_MAXIMUM (Kana) */
23. 0x91, 0x02, /* OUTPUT (Data,Var,Abs) */
24. 0x95, 0x01, /* REPORT_COUNT (1) */
25. 0x75, 0x03, /* REPORT_SIZE (3) */
26. 0x91, 0x01, /* OUTPUT (Cnst,Var,Abs) */
27. 0x95, 0x06, /* REPORT_COUNT (6) */
28. 0x75, 0x08, /* REPORT_SIZE (8) */
29. 0x15, 0x00, /* LOGICAL_MINIMUM (0) */
30. 0x25, 0xa4, /* LOGICAL_MAXIMUM (164) */ /* Can be 255 */
31. 0x05, 0x07, /* USAGE_PAGE (Keyboard) */
32. 0x19, 0x00, /* USAGE_MINIMUM (Reserved-no event indicated) */
33. 0x29, 0xa4, /* USAGE_MAXIMUM (Keyboard Application) */ /* Can be 255 */
34. 0x81, 0x00, /* INPUT (Data,Ary,Abs) */
35. 0xc0, /* END_COLLECTION */
36. #ifdef MULTIMEDIA_KEYBOARD
37. 0x05, 0x0c, /* USAGE_PAGE (Consumer) */
38. 0x09, 0x01, /* USAGE (Consumer Control) */
39. 0xa1, 0x01, /* COLLECTION (Application) */
40. 0x85, 0x04, /* REPORT_ID (4) */
41. 0x15, 0x00, /* LOGICAL_MINIMUM (0) */
42. 0x25, 0x01, /* LOGICAL_MAXIMUM (1) */
43. 0x75, 0x01, /* REPORT_SIZE (1) */
44. 0x95, 0x18, /* REPORT_COUNT (24) */
45. 0x09, 0xb5, /* USAGE (Scan Next Track) */
46. 0x09, 0xb6, /* USAGE (Scan Previous Track) */
47. 0x09, 0xb7, /* USAGE (Stop) */
48. 0x09, 0xcd, /* USAGE (Play/Pause) */
49. 0x09, 0xe2, /* USAGE (Mute) */
50. 0x09, 0xe5, /* USAGE (Bass Boost) */
51. 0x09, 0xe7, /* USAGE (Loudness) */
52. 0x09, 0xe9, /* USAGE (Volume Increment) */
53. 0x09, 0xea, /* USAGE (Volume Decrement) */
54. 0x0a, 0x52, 0x01, /* USAGE (Bass Increment) */

```

```

55.    0x0a, 0x53, 0x01,    /* USAGE      (Bass Decrement) */
56.    0x0a, 0x54, 0x01,    /* USAGE      (Treble Increment) */
57.    0x0a, 0x55, 0x01,    /* USAGE      (Treble Decrement) */
58.    0x0a, 0x83, 0x01,    /* USAGE      (AL Consumer Control Configuration) */
59.    0x0a, 0x8a, 0x01,    /* USAGE      (AL Email Reader) */
60.    0x0a, 0x92, 0x01,    /* USAGE      (AL Calculator) */
61.    0x0a, 0x94, 0x01,    /* USAGE      (AL Local Machine Browser) */
62.    0x0a, 0x21, 0x02,    /* USAGE      (AC Search) */
63.    0x0a, 0x23, 0x02,    /* USAGE      (AC Home) */
64.    0x0a, 0x24, 0x02,    /* USAGE      (AC Back) */
65.    0x0a, 0x25, 0x02,    /* USAGE      (AC Forward) */
66.    0x0a, 0x26, 0x02,    /* USAGE      (AC Stop) */
67.    0x0a, 0x27, 0x02,    /* USAGE      (AC Refresh) */
68.    0x0a, 0x2a, 0x02,    /* USAGE      (AC Bookmarks) */
69.    0x81, 0x02,          /* INPUT      (Data,Var,Abs) */
70.    0xc0                /* END_COLLECTION */
71. #endif
72. };
73.

```

3.3 Advertising Data And Scan Response

SUT needs to perform search operation before connected with Keyboard, then obtain module name and service UUID from advertising data and scan response data. Advertising data and scan response data format are defined below:

```

1.  /** @brief GAP - scan response data (max size = 31 bytes) */
2.  static const uint8_t scan_rsp_data[] =
3.  {
4.      0x03,                /* length */
5.      GAP_ADTYPE_APPEARANCE, /* type="Appearance" */
6.      LO_WORD(GAP_GATT_APPEARANCE_KEYBOARD),
7.      HI_WORD(GAP_GATT_APPEARANCE_KEYBOARD),
8.  };
9.
10. /** @brief GAP - Advertisement data (max size = 31 bytes, best kept short to conserve
    power) */
11. static const uint8_t adv_data[] =
12. {
13.     /* Flags */
14.     0x02,                /* length */
15.     GAP_ADTYPE_FLAGS, /* type="Flags" */

```

```
16.    GAP_ADTYPE_FLAGS_LIMITED | GAP_ADTYPE_FLAGS_BREDR_NOT_SUPPORTED,  
17.    /* Service */  
18.    0x03,          /* length */  
19.    GAP_ADTYPE_16BIT_COMPLETE,  
20.    0x12,  
21.    0x18,  
22.    /* Local name */  
23.    0x0D,          /* length */  
24.    GAP_ADTYPE_LOCAL_NAME_COMPLETE,  
25.    'B', 'L', 'E', '_', 'K', 'E', 'Y', 'B', 'O', 'A', 'R', 'D'  
26. };
```

4 Reference

[1] IEEE Std 11073-20601™- 2008 Health Informatics - Personal Health Device Communication - Application Profile - Optimized Exchange Protocol - version 1.0 or later.